

Day 4 Problem 1

SUBMISSION BY :

APURV JHA

Q1 · Arrays

Second Largest Without Duplicates

Given an integer array, find and return the second largest unique element. If no such element exists, return -1.

Input: [12, 35, 1, 10, 34, 1]

Output: 34

Input: [10, 10, 10]

Output: -1

Constraint: Do not use sorting. O(n) solution expected.

Easy~10 min Linear traversal

```
Day4 > Problem 1 > SecondLargestElement.cpp > main()
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> arr = {12, 35, 1, 10, 34, 1};
7
8      int largest = arr[0];
9
10     for (int i = 1; i < arr.size(); i++) {
11         if (arr[i] > largest) {
12             largest = arr[i];
13         }
14     }
15
16     int secondLargest = -1;
17
18     for (int i = 0; i < arr.size(); i++) {
19         if (arr[i] != largest && arr[i] > secondLargest) {
20             secondLargest = arr[i];
21         }
22     }
23
24     cout << secondLargest;
25
26     return 0;
27 }
```

Q2 · Arrays

Rotate Array K Steps to the Right

Given an array of integers and a number k, rotate the array to the right by k positions in-place.

Input: [1, 2, 3, 4, 5, 6, 7], k = 3

Output: [5, 6, 7, 1, 2, 3, 4]

Input: [-1, -100, 3, 99], k = 2

Output: [3, 99, -1, -100]

Handle edge case: k may be larger than array length.

Medium~12 min In-place, modulo

```
Day4 > Problem 1 > K_Rotation.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      int arr[n];
9
10     for (int i = 0; i < n; i++) {
11         cin >> arr[i];
12     }
13
14     int k;
15     cin >> k;
16
17     k = k % n;
18
19     for (int r = 0; r < k; r++) {
20         int last = arr[n - 1];
21
22         for (int i = n - 1; i > 0; i--) {
23             arr[i] = arr[i - 1];
24         }
25
26         arr[0] = last;
27     }
28
29     for (int i = 0; i < n; i++) {
30         cout << arr[i] << " ";
31     }
32
33     return 0;
34 }
```

Q3 · Strings

Valid Anagram Check

Write a function that takes two strings and returns true if one is an anagram of the other, false otherwise. Comparison should be case-insensitive and ignore spaces.

Input: "Listen", "Silent"

Output: true

Input: "hello world", "world hello"

Output: true

Input: "rat", "car"

Output: false

Do not use any built-in sort function. Use a frequency map.

Easy~8 minHash map

```
Day4 > Problem 1 > C++ AnagramCheck.cpp > main()
1  #include <iostream>
2  #include <unordered_map>
3  #include <string>
4  using namespace std;
5
6  bool isAnagram(string s1, string s2) {
7      unordered_map<char, int> mp;
8
9      for (char c : s1) {
10         if (c != ' ') {
11             c = tolower(c);
12             mp[c]++;
13         }
14     }
15
16     for (char c : s2) {
17         if (c != ' ') {
18             c = tolower(c);
19             mp[c]--;
20         }
21     }
22
23     for (auto x : mp) {
24         if (x.second != 0)
25             return false;
26     }
27
28     return true;
29 }
30
31 int main() {
32     string s1, s2;
33
34     getline(cin, s1);
35     getline(cin, s2);
36
37     cout << (isAnagram(s1, s2) ? "true" : "false");
38
39     return 0;
40 }
```

Q4 · Strings

Longest Substring Without Repeating Characters

Given a string, find the length of the longest substring that contains no repeating characters.

Input: "abcabcbbb"

Output: 3 (substring: "abc")

Input: "bbbbbb"

Output: 1 (substring: "b")

Input: "pwwkew"

Output: 3 (substring: "wke")

Sliding window approach expected. Brute force will receive partial credit.

Medium~12 min Sliding window

```
Day4 > Problem 1 > LongestSubstring.cpp > main()
1  #include <iostream>
2  #include <unordered_map>
3  #include <string>
4  using namespace std;
5
6  int main() {
7      string s;
8      cin >> s;
9
10     unordered_map<char, int> mp;
11
12     int start = 0;
13     int maxLen = 0;
14
15     for (int end = 0; end < s.length(); end++) {
16         if (mp.find(s[end]) != mp.end() && mp[s[end]] >= start) {
17             start = mp[s[end]] + 1;
18         }
19
20         mp[s[end]] = end;
21
22         int len = end - start + 1;
23
24         if (len > maxLen) {
25             maxLen = len;
26         }
27     }
28
29     cout << maxLen;
30
31     return 0;
32 }
```

Q5 · SQL

Top Earning Employees per Department

Given the tables below, write a query to find the name and salary of the highest-paid employee in each department. Include department name in the result.

Employees(emp_id, name, salary, dept_id)

Departments(dept_id, dept_name)

Expected output columns: dept_name, name, salary

Handle the case where two employees share the top salary in the same department — both should appear.

Medium~10 min JOIN, GROUP BY

Solution:

```
SELECT d.dept_name,e.name,e.salary
FROM Employees e
JOIN Departments d
ON e.dept_id = d.dept_id
WHERE e.salary = (
    SELECT MAX(salary)
    FROM Employees
    WHERE dept_id = e.dept_id
);
```

Q6 · SQL

Monthly Order Revenue Report

Using the Orders table, write a query to find the total revenue and number of orders placed each month in the year 2024, ordered from highest revenue to lowest.

Orders(order_id, customer_id, order_date, amount)

Expected output columns:

month, total_revenue, total_orders

Use DATE functions to extract the month. Format the month as a readable name (Jan, Feb...) if your SQL dialect supports it.

Easy–Medium~8 minAggregate, date functions

Solution:

```
SELECT MONTHNAME(order_date) AS month,
    SUM(amount) AS total_revenue,
    COUNT(order_id) AS total_orders
FROM Orders
WHERE YEAR(order_date) = 2024
GROUP BY MONTH(order_date), MONTHNAME(order_date)
ORDER BY total_revenue DESC;
```